



# Inl 1 - Biljettsystem

## Information

- Betygsskala: IG/G
- Deadline: fredagen den 18 september 2026 kl. 23.59
- Mål från kursplanen som examineras:
  - (1) Redogöra för CORS
  - (2) Använda backend med hjälp av Node.js
  - (3) Förklara hur Node.js och en NoSQL-databas kan samarbeta tillsammans
- Inlämning:
  1. Commit:a *och pusha* koden till ditt repo på GitHub. Verifiera att det du har på datorn även syns på GitHub.
  2. Se till att repot är publikt och/eller bjud in användaren `postmodernistx` som en "collaborator" till repot.
  3. Klistra in länken till repot på itslearning i inlämningsboxen.

## Uppgiftsbeskrivning

CORS är ett av dom bökigaste koncepten inom webbutveckling, som nästan alltid orsakar huvudbry.

I den här uppgiften ska vi fokusera på just CORS, och bygga ett fullstack-monorepo som innehåller både front end och back end.

### ▼ Syftet med uppgiften

Du lär dig att:

- Förstå och konfigurera CORS

- Skapa en enkel "template" som du kan utgå ifrån i kommande projekt
- Integrera hela JavaScript-ekosystemet (front end, back end, NoSQL-databaser)

## Att göra

I uppgiften ska du bygga ett biljettsystem. Biljettsystemet ska ha följande egenskaper:

- Skapa en ny biljett (generera en slumpmässig kod)
- Använda en biljett genom att mata in koden (den ska inte gå att använda igen)
- Radera en biljett som inte ännu är använd
- Lista biljetter som finns i systemet, och om de är använda eller inte

I projektet ska det finnas ett (mycket enkelt) front end för att en användare ska kunna utföra dessa steg.

Och det ska finnas ett motsvarande backend och en databas som hanterar biljetterna.

I projektet kommer vi att använda en lokal databas i projektet, t.ex. better-sqlite3.

Glöm inte att projektet är ett IG/G-projekt.

### ▼ Steg 1. Wireframe

Projekt är alltid lättast att sätta igång om man har en vision för vad slutprodukten ska vara. Börja med att skissa en enkel wireframe för ditt frontend, på t.ex. papper. Du behöver inte lämna in din wireframe, utan den är ett stöd för dig.

### ▼ Steg 2. Endpoints

- Vilka endpoint behöver du för de olika funktionerna som ska finnas med i applikationen?
- Vilken säkerhet behöver implementeras, t.ex. vem har rätt att radera biljetter?

### ▼ Steg 3. Databasdesign

Tänk igenom hurdan din databas behöver se ut för att hantera biljetterna. Sätt upp script för att generera databasen.

#### ▼ Steg 4. TDD

Välj vilken endpoint du vill börja med, t.ex. för att skapa en ny biljett.

Skriv ett test för backend och ett test för frontend. Se till att båda testerna misslyckas.

Exempelidé: Du har en knapp som är "Skapa biljett", den skickar en request till backend som genererar en kod. Informationen lagras i databasen (kod, tidsstämpel, om koden är använd eller inte, ev. giltighetstid). Svaret/responsen skickas till front end (den genererade koden och övrig info du behöver/vill ha med).

Försök att skriva *testerna* först, och först därefter koden. Referera till "red, green, refactor"-metoden.

#### ▼ Steg 5. Första endpointen & front end

Börja sedan skriva på din kod så att testerna successivt går igenom, failar, går igenom igen, osv.

Iterera fram din kod tills hela flödet fungerar som tänkt.

---

## Kravlista

- Ett fungerande front end för att lösa uppgifterna (du behöver inte göra någon snygg CSS, det är upp till dig). Du väljer själv ramverk (Vue, React, Angular, nåt annat - men ett ramverk, ej endast HTML/JS).
- Ett fungerande back end
- En databasdesign
- Du skickar in ett kodproblem som du har stött på, och din lösning på det i "frågelådan" (glöm inte att skriva ditt namn)

---

## Redovisning

- I inlämningen ska du ha med en README som innehåller:
  - En kort beskrivning på projektet
  - En bild på din databasdesign (textbaserad eller i bildformat)
  - Instruktioner för hur man kommer igång med projektet
- Redovisning i klass
  - 2-3 minuter per person
  - Visa upp ditt front end, hur det blev

- Nämn en issue som du stötte på och hur du löste det (kan vara samma som du skickat in i frågelådan)



## Resurser

- [better-sqlite3](#) ↗
- [Red, green, refactor](#) ↗

## Bedömningsexempel

### För IG

- Uppgiften är inte utförd enligt instruktionerna

### För G

- Uppgiften följer kravlistan och har alla delar implementerande

## Vanliga misstag

### ▼ Works on my machine! (Portar)

Problem: Olika portar är ofta ett problem, t.ex. att front end körs på `:3000` och backend på `:5000` .

Exempel: Du ( `:3000` ) försöker köpa en apelsin i mataffären, men du får inte komma in i mataffären ( `:5000` ) för att du har blivit "portad". Mataffären har helt enkelt glömt att lägga till dig över "godkända besökare".

Lösning: Använd ett `cors()` -middleware i Express och se till att frontend-porten är tillåten.

### ▼ Dev vs. prod

Problem: Det funkar bra i dev, men du glömde lägga till produktionsadressen

Exempel: Du tillåter `localhost` att köra, men när du väl flyttar projektet till produktion ( `my-cool-site.com` ), så fungerar det inte längre.

Lösning: Lägg till en `.env` -fil så att den automatiskt växlar mellan prod och dev. T.ex. `.env.prod` och `.env.dev` .

#### ▼ Säkerhet

Problem: Du försöker skicka en special-header eller cookie, men backend tillåter endast standard-förfrågningar.

Lösning: Konfigurera CORS att använda specifika headers, t.ex. `allowedHeaders: ['Content-Type', 'Authorization']` eller `credentials: true` .

#### ▼ Pre-flight check

Innan man skickar t.ex. `POST` eller `DELETE` så kollar webbläsaren `OPTIONS` , för att checka att backend-dörren är öppen. Om inte backend kan hantera `OPTIONS` så kommer hela förfarandet att misslyckas.

Lösning: Se till att Express-servern svarar ordentligt på HTTP OPTIONS.

#### ▼ Slash i adressen

Ibland är det känsligt att det ska finnas med en avslutande `/` i anropet.

Jämför:

- `http://localhost:5000/api/get-tickets/` (slash i slutet)
- `http://localhost:5000/api/get-tickets` (utan slash)

## ? Våga fråga & tips

Här kan du ställa frågor, eller tipsa om saker! Sharing is caring :)

► Öppna frågelådan



Modulen är inte klar.

Markera modulen som klar

Senast uppdaterad: Monday, August 3, 2026 at 9:34 PM

---

Föregående sida

Handledningsschema

Nästa sida

 Inl 2 - API